



Программирование на С и С++, принципы работы UNIX систем

Лабораторная работа №2

«Компиляция исходного кода. Сравнение С++ и Python»

Преподаватель:

Маслов И. Д.

Студент:

Бедь В. В.

1 Цель работы

Познакомиться с принципами компиляции исходного кода. Составить программу с использованием циклов, условий и функций. Сравнить быстродействие между C++ и Python. Ознакомиться с типами данных.

2 Ход выполнения работы

2.1 Программа на C++

Был создан файл src/main.cpp со следующим содержанием:

```
1  #include <iostream>
2  #include <chrono>
3  #include <limits>
4
5  using namespace std;
6  using namespace chrono;
7
8  //                                     : x^2 - x^2 + x*4 - x*5
9  + x + x
10 double compute(double x) {
11     return x * x - x * x + x * 4 - x * 5 + x + x;
12 }
13
14 int main() {
15     while (true) {
16         cout << "Enter number of iterations: ";
17
18         long long n;
19         if (!(cin >> n)) {
20             cout << "Invalid input. Exiting program.\n";
21             break;
22         }
23
24         cout << "Starting calculation with " << n << " iterations...\n";
25
26         auto start = high_resolution_clock::now();
27
28         double result = 0;
29         for (long long i = 0; i < n; i++) {
30             result = compute(2);
31         }
32
33         auto end = high_resolution_clock::now();
34         auto duration = duration_cast<milliseconds>(end - start).count();
35
36         cout << "Time taken: " << duration << " ms\n";
37         cout << "Last computed value: " << result << "\n\n";
38     }
39
40     return 0;
41 }
```

Листинг 1: src/main.cpp

2.2 Программа на Python 3

Был создан файл `src/main.py` со следующим содержимым:

```
1 import time
2
3 def compute(x):
4     return x*x - x*x + x*4 - x*5 + x + x
5
6 def main():
7     while True:
8         try:
9             n = input("Enter number of iterations: ")
10            n = int(n)
11        except ValueError:
12            print("Invalid input. Exiting program.")
13            break
14
15        print(f"Starting calculation with {n} iterations...")
16
17        start = time.time()
18
19        result = 0
20        for _ in range(n):
21            result = compute(2)
22
23        end = time.time()
24        elapsed_ms = (end - start) * 1000
25
26        print(f"Time taken: {elapsed_ms:.2f} ms")
27        print(f>Last computed value: {result}\n")
28
29 if __name__ == "__main__":
30     main()
```

Листинг 2: `src/main.py`

2.3 Особенности вычисляемого выражения

Выражение $x^2 - x^2 + 4x - 5x + x + x$ математически упрощается до x :

$$x^2 - x^2 + 4x - 5x + x + x = (x^2 - x^2) + (4x - 5x + x + x) = 0 + x = x$$

Это свойство было использовано для демонстрации разницы между компилируемыми и интерпретируемыми языками:

- **C++** на этапе компиляции с оптимизацией (`-O2`) упрощает выражение и может полностью удалить бесполезный цикл.
- **Python** выполняет все операции буквально на каждой итерации, что наглядно показывает разницу в производительности.

3 Настройка репозитория и CI/CD

3.1 Создание .gitignore

Для игнорирования папок сборки и временных файлов был создан файл .gitignore:

```
1 #
2 build/
3 build_stg/
4 build_prd/
5
6 #
7 *.exe
8 *.o
9 *.out
10 *.obj
11
12 # Python
13 __pycache__/*
14 *.pyc
15 *.pyo
16
17 #
18 .vscode/
19 .idea/
20 *.swp
21 *.swo
22 *~
23
24 #
25 *.log
26 *.tmp
```

Листинг 3: .gitignore

3.2 Создание скриптов наката ревизий

Для автоматического развертывания ревизий на окружения STG (staging) и PRD (production) были созданы скрипты.

3.2.1 Скрипт deploy_to_stg.sh

```
1 #!/bin/bash
2 echo "Deploying to STG..."
3 mkdir -p build_stg
4 g++ src/main.cpp -o build_stg/cpp_program -O2
5 cp src/main.py build_stg/
6 echo "STG deployment done. Files in ./build_stg/"
```

Листинг 4: scripts/deploy_to_stg.sh

3.2.2 Скрипт deploy_to_prd.sh

```
1 #!/bin/bash
2 echo "Deploying to PRD..."
3 mkdir -p build_prd
4 g++ src/main.cpp -o build_prd/cpp_program -O2
5 cp src/main.py build_prd/
6 echo "PRD deployment done. Files in ./build_prd/"
```

Листинг 5: scripts/deploy_to_prd.sh

3.3 Компиляция и запуск

Для компиляции и запуска программ использовались следующие команды в ОС Linux (WSL):

```
1 # STG ( )
2 bash scripts/deploy_to_stg.sh
3
4 #
5 cd build_stg
6
7 # C++
8 ./cpp_program
9
10 # Python
11 python3 main.py
12
13 # PRD
14 cd ~/cpp_unix_labs
15 bash scripts/deploy_to_prd.sh
```

Листинг 6: Компиляция и запуск

3.4 Фиксация ревизий в Git

Все изменения фиксировались в ветке dev:

```
1 git checkout dev
2 git add src/main.cpp src/main.py .gitignore scripts/
3 git commit -m "Add C++ and Python programs for lab2"
4 git commit -m "Add .gitignore and deployment scripts"
```

Листинг 7: Фиксация ревизий

4 Результаты тестирования

Тестирование проводилось для 10^7 итераций ($n = 10\,000\,000$) на идентичном оборудовании.

Язык	Время выполнения (мс)	Примечание
C++	≈ 0	Цикл оптимизирован компилятором
Python 3	≈ 2235	Интерпретатор выполняет все операции

Таблица 1: Сравнение производительности C++ и Python

5 Анализ результатов

Полученные результаты демонстрируют фундаментальную разницу между компилируемыми и интерпретируемыми языками:

1. **C++:** Компилятор анализирует код, упрощает выражение $x^2 - x^2 + 4x - 5x + x + x$ до x и, обнаружив, что результат не используется, полностью удаляет цикл. Время выполнения стремится к нулю.
2. **Python:** Интерпретатор на каждой итерации выполняет лексический и синтаксический анализ, компиляцию в байт-код и его интерпретацию. Динамическая типизация добавляет дополнительные накладные расходы. Время выполнения значительно выше.

6 Вывод

В ходе выполнения лабораторной работы были получены следующие результаты:

- Разработаны программы на C++ и Python, реализующие циклическое вычисление заданного выражения.
- Продемонстрирована разница в производительности: Python медленнее C++ в десятки раз для данного типа вычислений.
- Создана инфраструктура CI/CD: настроен Git с веткой `dev`, созданы скрипты наката ревизий на окружения STG и PRD.
- Папка `build/` добавлена в `.gitignore`, что предотвращает попадание бинарных файлов в репозиторий.

7 Ссылка на репозиторий

Тут